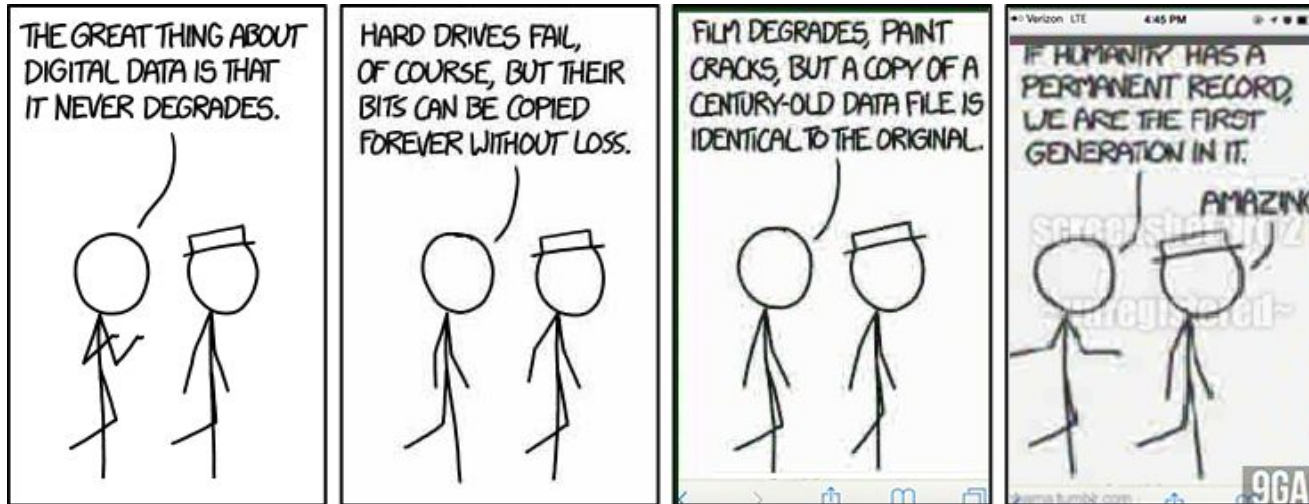


Διάλεξη #16 - Integrity

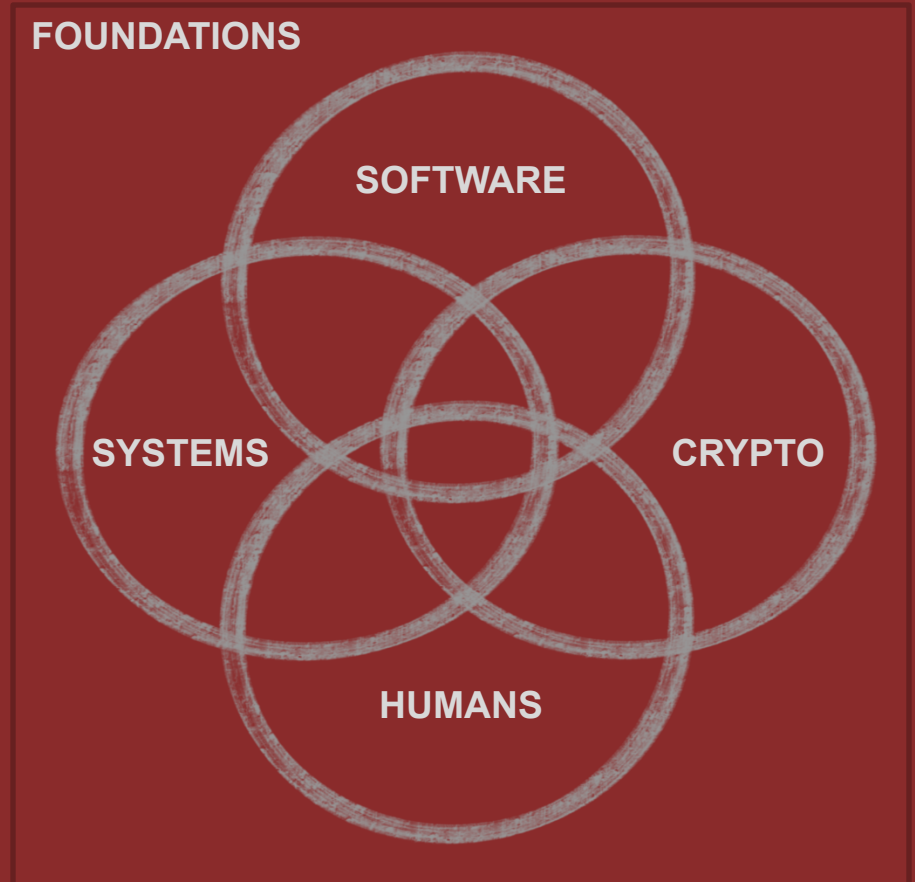
Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών

Εισαγωγή στην Ασφάλεια

Θανάσης Αυγερινός



Huge thank you to [David Brumley](#) from Carnegie Mellon University for the guidance and content input while developing this class (lots of slides from Dan Boneh @ Stanford!)

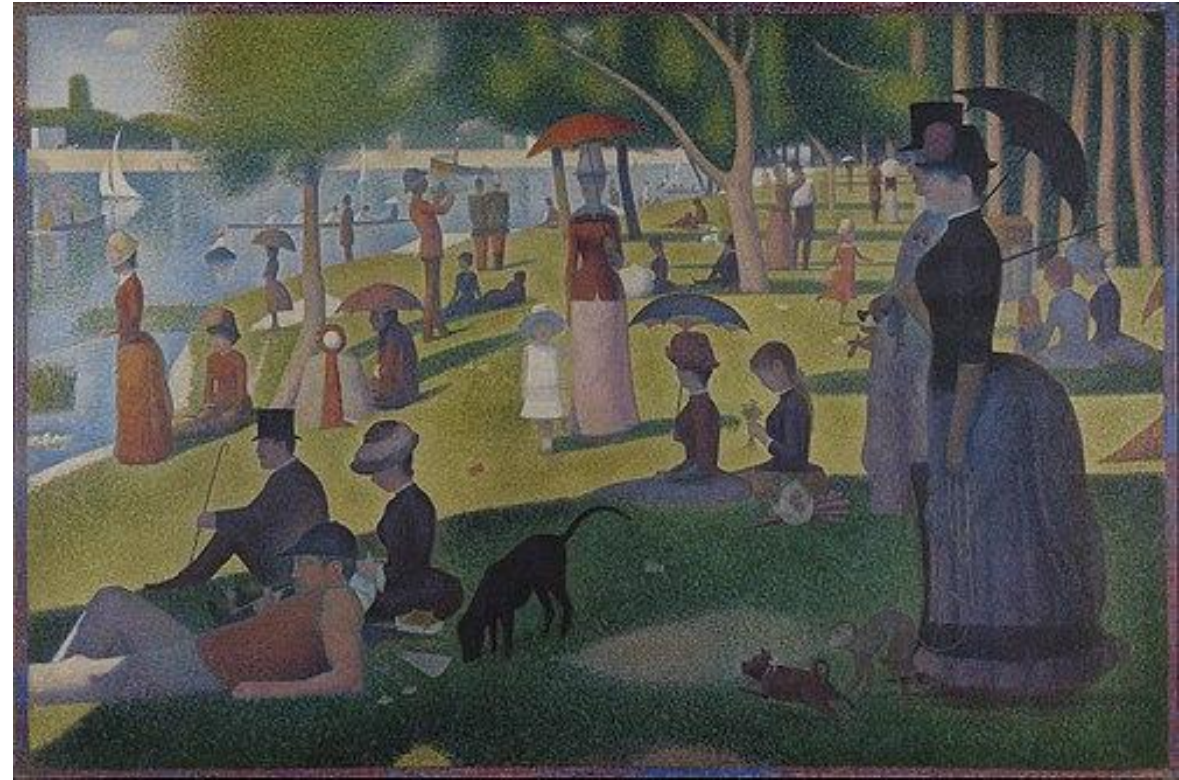


Ανακοινώσεις / Διευκρινίσεις

- Γιατί είχε K_1 , K ως κλειδί στο nonce-based MAC;
- Η ομαδική εργασία θα τρέξει 23-25 Μαΐου - any conflicts;

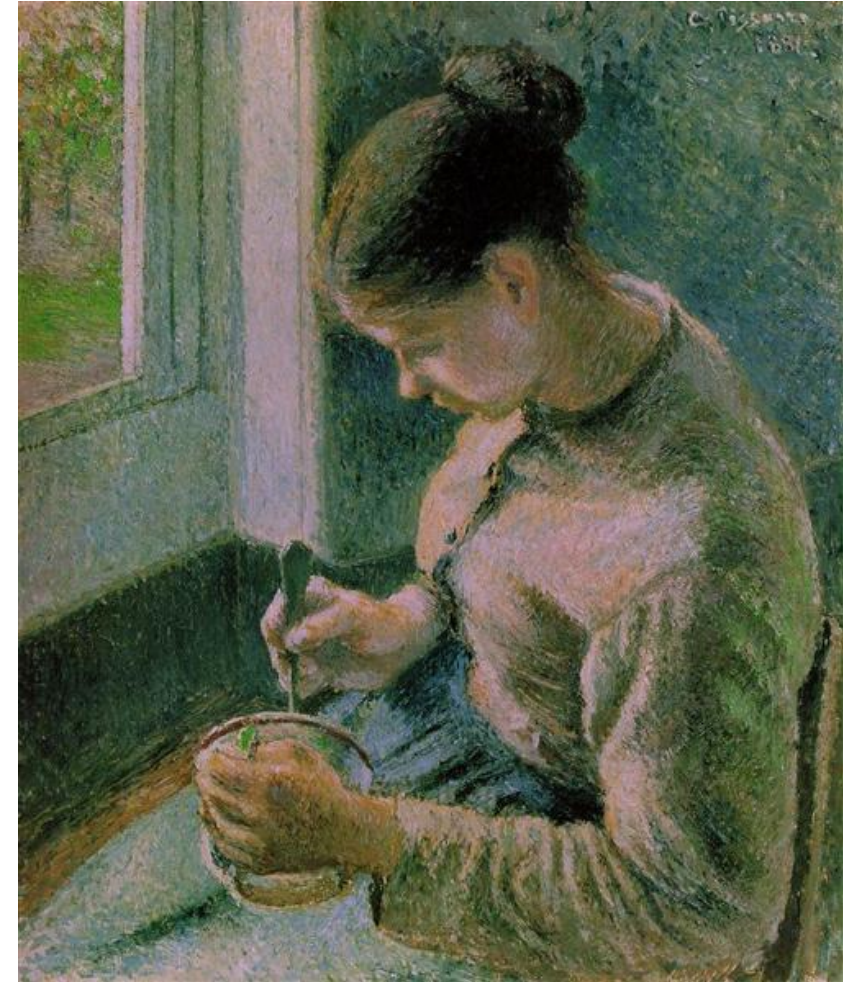
Την προηγούμενη φορά

- Encryption Modes
 - Electronic Code Book (ECB)
 - Cipher Block Chaining (CBC)
 - Counter Mode (CTR)
- Mistakes and Attacks



Σήμερα

- Message Integrity
 - Message Authentication Codes (MACs)
 - CBC-MAC, NMAC, CMAC
- Introduction to Hashing





Security in the News

'Copy Fail' Logic Flaw in Linux Kernel Enables System Takeover

Affecting the kernel's *authencsn* cryptographic template, the vulnerability was introduced in 2017 and impacts all distributions.

Tracked as [CVE-2026-31431](#) (CVSS score of 7.8) and dubbed [Copy Fail](#), the issue is believed to affect all Linux distributions since 2017.

The security defect impacts the kernel's *authencsn* Authenticated Encryption with Associated Data (AEAD) template, which IPsec uses for Extended Sequence Number (ESN) support.

According to Theori, the issue is that Linux places page cache pages in a writable scatterlist, that *authencsn* uses the caller's destination scatterlist as scratch space, and that a 2017 optimization put page cache pages in the writable scatterlist.

When performing byte rearrangement in the scratch space, *authencsn* makes a call that writes four bytes of code past the AEAD tag, into the cached copy of another file.

<https://copy.fail/>

Copy Fail allows an attacker with local code execution privileges to modify the in-memory copy of any setuid-root binary readable by the user, thus achieving root shell access, [Theori explains](#).



Block Cipher Attacks

Exhaustive Search for block cipher key

Goal: given a few input output pairs $(m_i, c_i = E(k, m_i))$ $i=1,..,3$
find key k .

Lemma: Suppose DES is an *ideal cipher*
(2^{56} random invertible functions)

Then $\forall m, c$ there is at most one key k s.t. $c = \text{DES}(k, m)$

Proof: $P[\exists k' \neq k : c = \text{DES}(k, m) = \text{DES}(k', m)] \leq$
 $\sum_{k' \in \{0,1\}^{56}} P[\text{DES}(k, m) = \text{DES}(k', m)] \leq 2^{56} \cdot \frac{1}{2^{64}} = \frac{1}{2^8}$ with prob. $\geq 1 - 1/256 \approx 99.5\%$

Exhaustive Search for block cipher key

For two DES pairs $(m_1, c_1 = \text{DES}(k, m_1))$, $(m_2, c_2 = \text{DES}(k, m_2))$
unicity prob. $\approx 1 - 1/2^{71}$

For AES-128: given two inp/out pairs, unicity prob. $\approx 1 - 1/2^{128}$

$\Rightarrow$ two input/output pairs are enough for exhaustive key search.

Strengthening DES against ex. search

Method 1: **Triple-DES**

- Let $E : K \times M \rightarrow M$ be a block cipher
- Define $\mathbf{3E} : K^3 \times M \rightarrow M$ as

$$\mathbf{3E}\left((k_1, k_2, k_3), m \right) = E(k_1, D(k_2, E(k_3, m)))$$

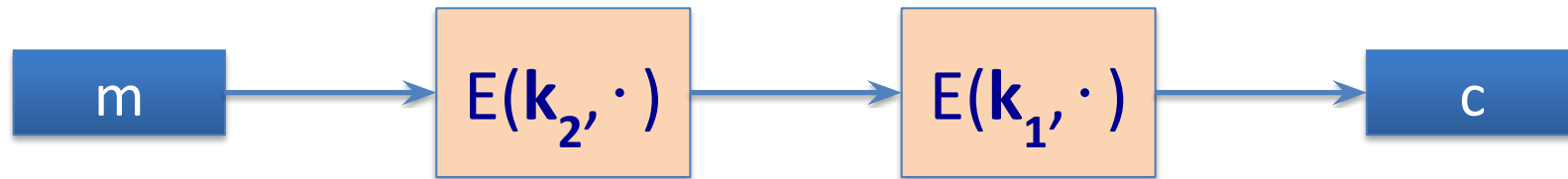
For 3DES: key-size = $3 \times 56 = 168$ bits. 3×slower than DES.

(simple attack in time $\approx 2^{112}$)

Why not double DES?

- Define $2E((k_1, k_2), m) = E(k_1, E(k_2, m))$

key-len = 112 bits for DES



Attack: $M = (m_1, \dots, m_{10})$, $C = (c_1, \dots, c_{10})$.

- step 1: build table.
sort on 2nd column

$k^0 = 00 \dots 00$	$E(k^0, M)$	} 2^{56} entries
$k^1 = 00 \dots 01$	$E(k^1, M)$	
$k^2 = 00 \dots 10$	$E(k^2, M)$	
$\vdots$	$\vdots$	
$k^N = 11 \dots 11$	$E(k^N, M)$	

Meet in the middle attack



Attack: $M = (m_1, \dots, m_{10})$, $C = (c_1, \dots, c_{10})$

- step 1: build table.
- Step 2: for all $k \in \{0,1\}^{56}$ do:
test if $D(k, C)$ is in 2^{nd} column.

if so then $E(k^i, M) = D(k, C) \Rightarrow (k^i, k) = (k_2, k_1)$

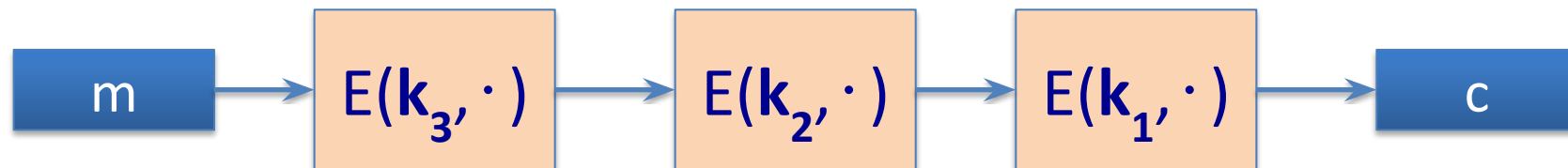
$k^0 = 00 \dots 00$	$E(k^0, M)$
$k^1 = 00 \dots 01$	$E(k^1, M)$
$k^2 = 00 \dots 10$	$E(k^2, M)$
$\vdots$	$\vdots$
$k^N = 11 \dots 11$	$E(k^N, M)$

Meet in the middle attack



$$\text{Time} = 2^{56} \log(2^{56}) + 2^{56} \log(2^{56}) < 2^{63} \ll 2^{112}, \quad \text{space} \approx 2^{56}$$

Same attack on 3DES: $\text{Time} = 2^{112}, \quad \text{space} \approx 2^{56}$



Method 2: DESX

$E : K \times \{0,1\}^n \rightarrow \{0,1\}^n$ a block cipher

Define EX as $EX((k_1, k_2, k_3), m) = k_1 \oplus E(k_2, m \oplus k_3)$

For DESX: key-len = 64+56+64 = 184 bits

... but easy attack in time $2^{64+56} = 2^{120}$

Note: $k_1 \oplus E(k_2, m)$ and $E(k_2, m \oplus k_1)$ does nothing !!

Quantum attacks

Generic search problem:

Let $f: X \rightarrow \{0,1\}$ be a function.

Goal: find $x \in X$ s.t. $f(x)=1$.

Classical computer: best generic algorithm time = $O(|X|)$

Quantum computer [[Grover '96](#)]: time = $O(|X|^{1/2})$

Can quantum computers be built: unknown

Quantum exhaustive search

Given $m, c=E(k,m)$ define

$$f(k) = \begin{cases} 1 & \text{if } E(k,m) = c \\ 0 & \text{otherwise} \end{cases}$$

Grover $\Rightarrow$ quantum computer can find k in time $O(|K|^{1/2})$

DES: time $\approx 2^{28}$, AES-128: time $\approx 2^{64}$

quantum computer $\Rightarrow$ 256-bits key ciphers (e.g. AES-256)

PRF Switching Lemma

Any secure PRP is also a secure PRF, if $|X|$ is sufficiently large.

Lemma: Let E be a PRP over (K, X)

Then for any q -query adversary A :

$$\left| \text{Adv}_{\text{PRF}}[A, E] - \text{Adv}_{\text{PRP}}[A, E] \right| < q^2 / 2|X|$$

$\Rightarrow$ Suppose $|X|$ is large so that $q^2 / 2|X|$ is “negligible”

Then $\text{Adv}_{\text{PRP}}[A, E]$ “negligible” $\Rightarrow \text{Adv}_{\text{PRF}}[A, E]$ “negligible”



Message Integrity

Message Integrity

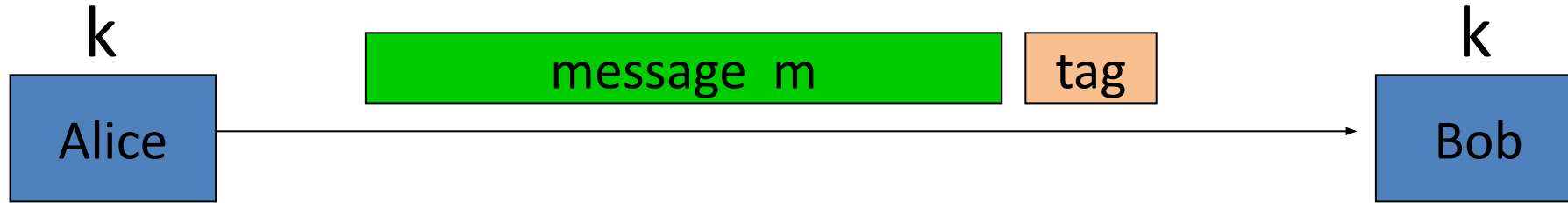
Goal: **integrity**, no confidentiality.

Examples:

- Transaction data / ledger.
- Communications.
- Public binaries on disk.
- Banner ads on web pages.



Message integrity: MACs



Generate tag (Sign):

$$\text{tag} \leftarrow S(k, m)$$

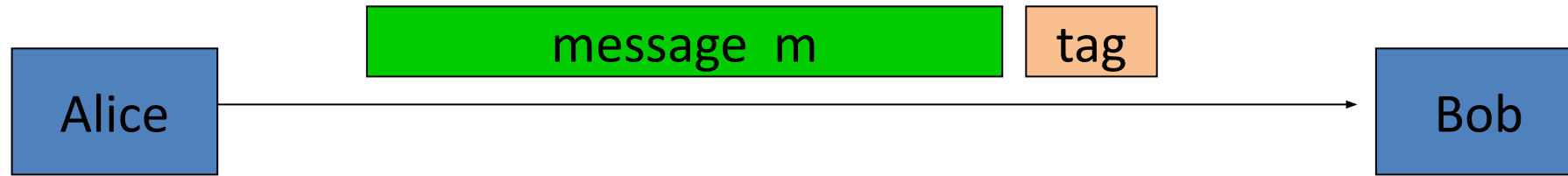
Verify tag:

$$V(k, m, \text{tag}) \stackrel{?}{=} \text{'yes'}$$

Def: **MAC** $I = (S, V)$ defined over (K, M, T) is a pair of algs:

- $S(k, m)$ outputs t in T
- $V(k, m, t)$ outputs 'yes' or 'no'

Integrity requires a secret key



Generate tag:

$\text{tag} \leftarrow \text{CRC}(m)$

Verify tag:

$V(m, \text{tag}) \stackrel{?}{=} \text{'yes'}$

- Attacker can easily modify message m and re-compute CRC.
- CRC designed to detect random, not malicious errors.

Secure MACs

Attacker's power: **chosen message attack**

- for $m_1, m_2, \dots, m_q$ attacker is given $t_i \leftarrow S(k, m_i)$

Attacker's goal: **existential forgery**

- produce some **new** valid message/tag pair (m, t) .

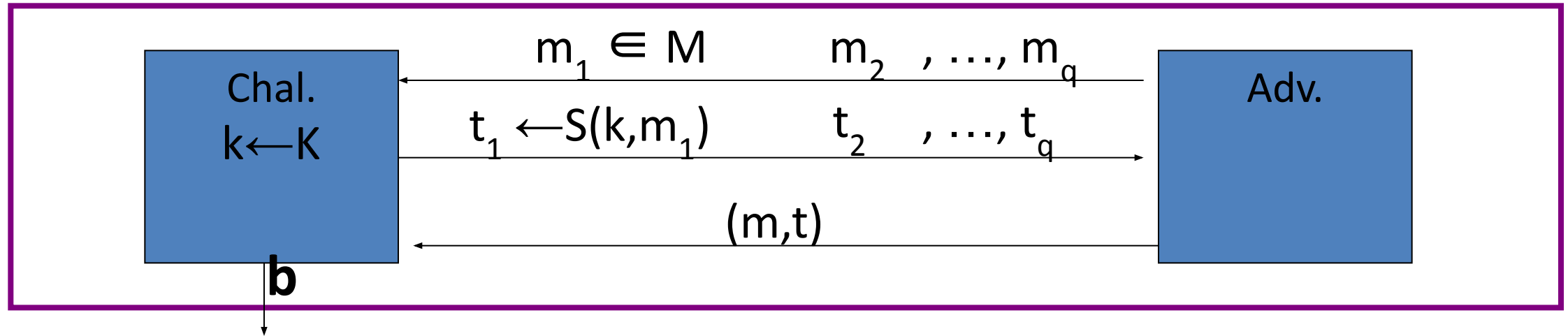
$$(m, t) \notin \{ (m_1, t_1), \dots, (m_q, t_q) \}$$

$\Rightarrow$ attacker cannot produce a valid tag for a new message

$\Rightarrow$ given (m, t) attacker cannot even produce (m, t') for $t' \neq t$

Secure MACs

- For a MAC $I=(S,V)$ and adv. A define a MAC game as:



$b=1$ if $V(k, m, t) = \text{'yes'}$ and $(m, t) \notin \{ (m_1, t_1), \dots, (m_q, t_q) \}$

$b=0$ otherwise

Def: $I=(S,V)$ is a **secure MAC** if for all “efficient” A :

$$\text{Adv}_{\text{MAC}}[A, I] = \Pr[\text{Chal. outputs } 1] \text{ is “negligible.”}$$

Let $I = (S, V)$ be a MAC.

Suppose an attacker is able to find $m_0 \neq m_1$ such that

$$S(k, m_0) = S(k, m_1) \quad \text{for } \frac{1}{2} \text{ of the keys } k \text{ in } K$$

Can this MAC be secure?

- ☐ Yes, the attacker cannot generate a valid tag for m_0 or m_1
- ☐ No, this MAC can be broken using a chosen msg attack
- ☐ It depends on the details of the MAC
- ☐

Let $I = (S,V)$ be a MAC.

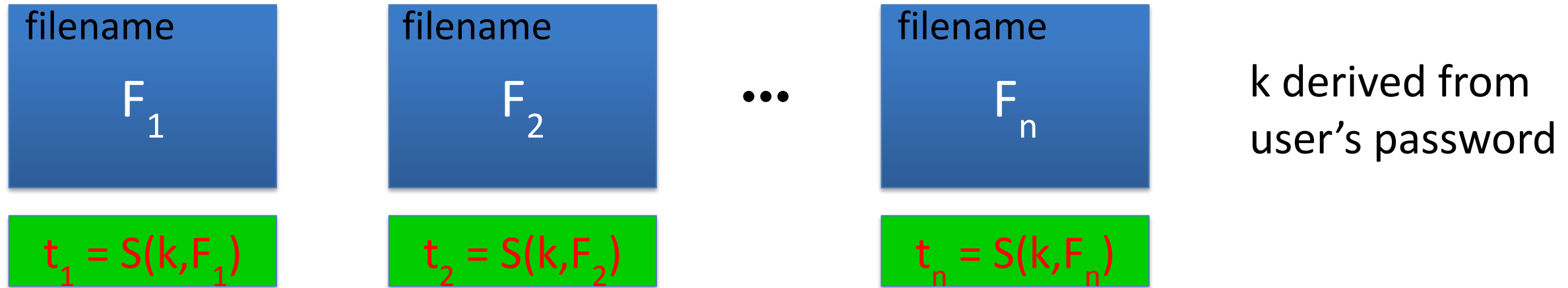
Suppose $S(k,m)$ is 5 bits long

Can this MAC be secure?

- ☐ No, an attacker can simply guess the tag for messages
- ☐ It depends on the details of the MAC
- ☐ Yes, the attacker cannot generate a valid tag for any message
- ☐

Example: protecting system files

Suppose at install time the system computes:



Later a virus infects system and modifies system files

User reboots into clean OS and supplies his password

– Then: secure MAC $\Rightarrow$ all modified files will be detected



Using PRFs to build MACs

Review: Secure MACs

MAC: signing alg. $S(k,m) \rightarrow t$ and verification alg. $V(k,m,t) \rightarrow 0,1$

Attacker's power: **chosen message attack**

- for $m_1, m_2, \dots, m_q$ attacker is given $t_i \leftarrow S(k, m_i)$

Attacker's goal: **existential forgery**

- produce some new valid message/tag pair (m,t) .

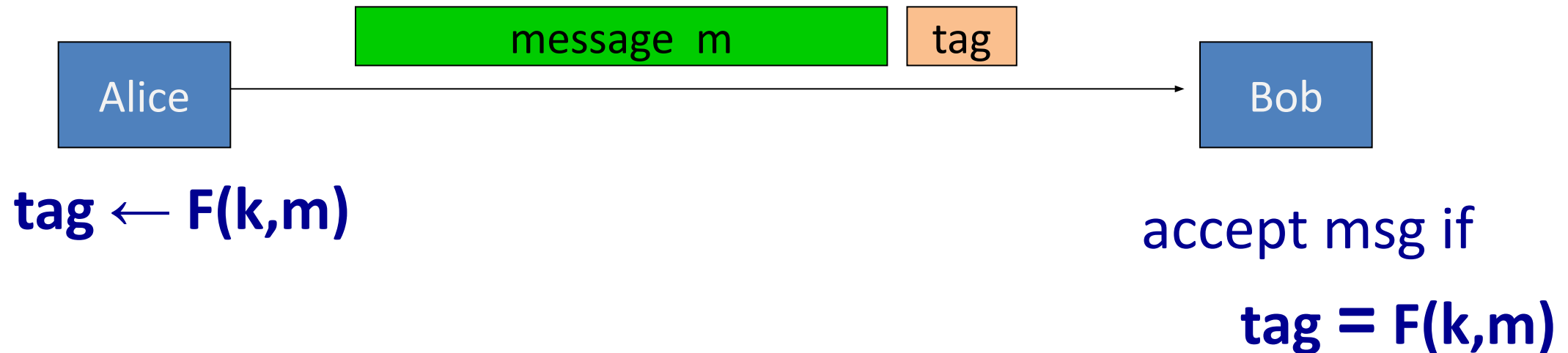
$$(m,t) \notin \{ (m_1, t_1), \dots, (m_q, t_q) \}$$

$\Rightarrow$ attacker cannot produce a valid tag for a new message

Secure PRF $\Rightarrow$ Secure MAC

For a PRF $F: K \times X \rightarrow Y$ define a MAC $I_F = (S, V)$ as:

- $S(k, m) := F(k, m)$
- $V(k, m, t)$: output 'yes' if $t = F(k, m)$ and 'no' otherwise.



A bad example

Suppose $F: K \times X \rightarrow Y$ is a secure PRF with $Y = \{0,1\}^{10}$

Is the derived MAC I_F a secure MAC system?

- ☐ Yes, the MAC is secure because the PRF is secure
- ☐ No tags are too short: anyone can guess the tag for any msg
- ☐ It depends on the function F
- ☐

Security

Thm: If $F: K \times X \rightarrow Y$ is a secure PRF and $1/|Y|$ is negligible (i.e. $|Y|$ is large) then I_F is a secure MAC.

In particular, for every eff. MAC adversary A attacking I_F there exists an eff. PRF adversary B attacking F s.t.:

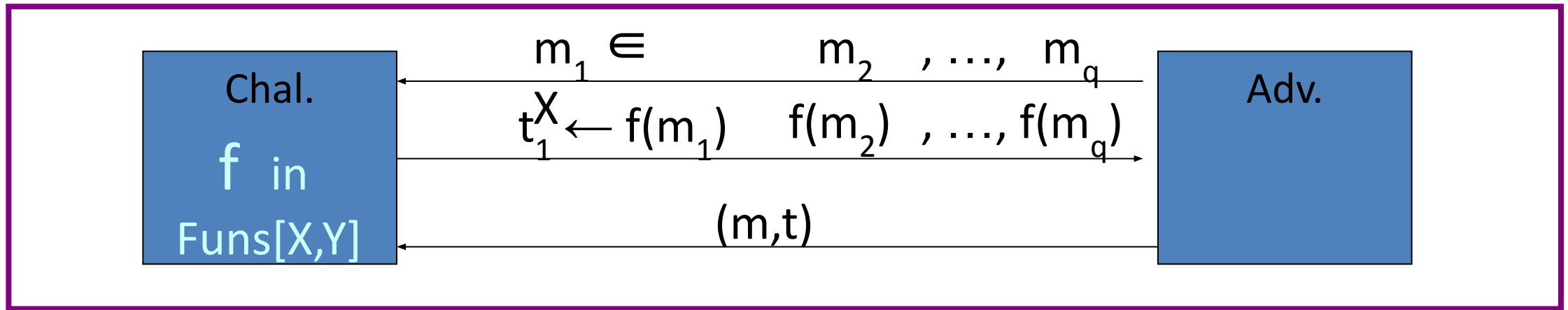
$$\text{Adv}_{\text{MAC}}[A, I_F] \leq \text{Adv}_{\text{PRF}}[B, F] + 1/|Y|$$

$\Rightarrow I_F$ is secure as long as $|Y|$ is large, say $|Y| = 2^{128}$.

Proof Sketch

Suppose $f: X \rightarrow Y$ is a truly random function

Then MAC adversary A must win the following game:



A wins if $t = f(m)$ and $m \notin \{m_1, \dots, m_q\}$

$\Rightarrow \Pr[A \text{ wins}] = 1/|Y|$

same must hold for $F(k, x)$

Examples

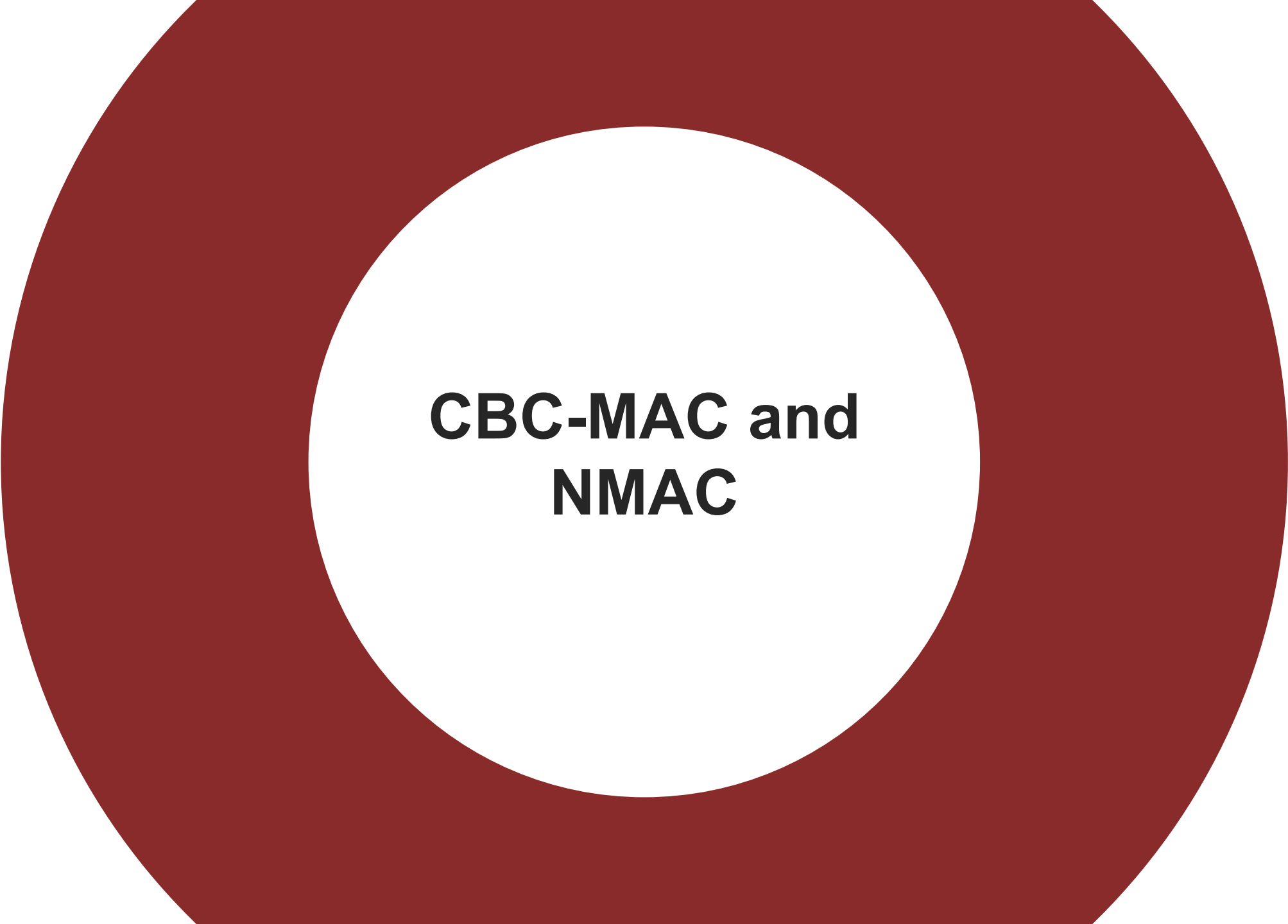
- AES: a MAC for 16-byte messages.
- Main question: how to convert Small-MAC into a Big-MAC ?
- Two main constructions used in practice:
 - **CBC-MAC** (banking – ANSI X9.9, X9.19, FIPS 186-3)
 - **HMAC** (Internet protocols: SSL, IPsec, SSH, ...)
- Both convert a small-PRF into a big-PRF.

Truncating MACs based on PRFs

Easy lemma: suppose $F: K \times X \rightarrow \{0,1\}^n$ is a secure PRF.

Then so is $F_t(k,m) = F(k,m)[1\dots t]$ for all $1 \leq t \leq n$

$\Rightarrow$ if (S,V) is a MAC is based on a secure PRF outputting n -bit tags
the truncated MAC outputting w bits is secure
... as long as $1/2^w$ is still negligible (say $w \geq 64$)



CBC-MAC and NMAC

MACs and PRFs

Recall: secure PRF $\mathbf{F} \Rightarrow$ secure MAC, as long as $|Y|$ is large

$$S(k, m) = F(k, m)$$

Our goal:

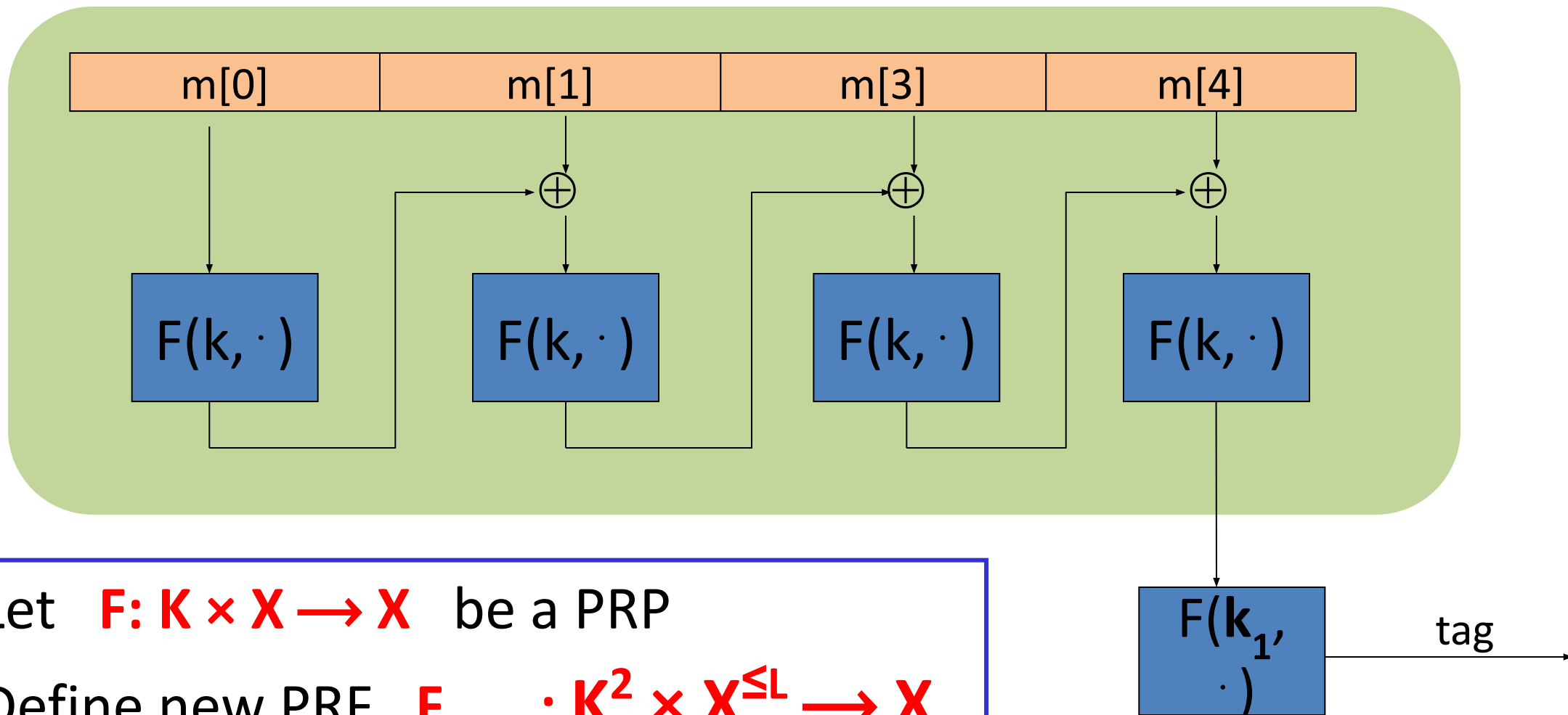
given a PRF for short messages (AES)

construct a PRF for long messages

From here on let $X = \{0,1\}^n$ (e.g. $n=128$)

Construction 1: encrypted CBC-MAC

raw CBC

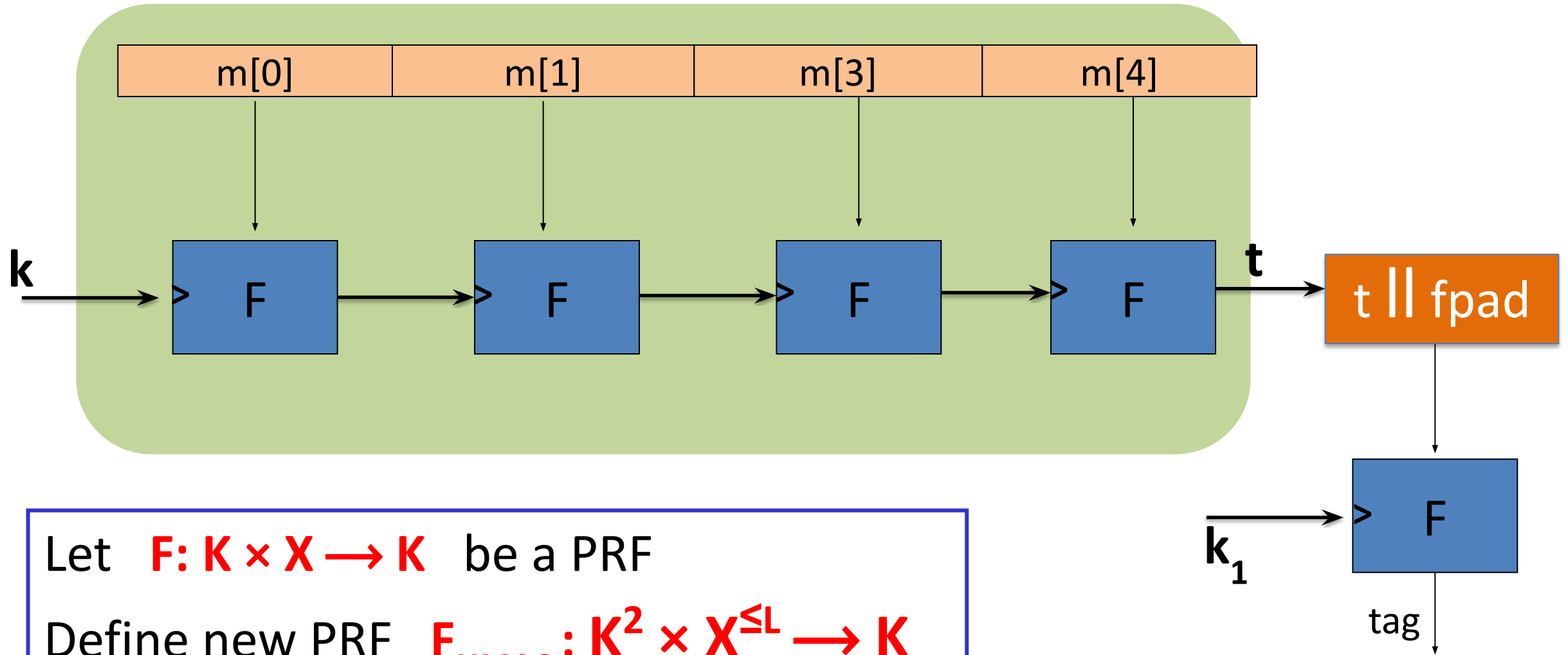


Let $F: K \times X \rightarrow X$ be a PRP

Define new PRF $F_{\text{ECBC}}: K^2 \times X^{\leq L} \rightarrow X$

Construction 2: NMAC (nested MAC)

cascade



Let $F: K \times X \rightarrow K$ be a PRF

Define new PRF $F_{\text{NMAC}}: K^2 \times X^{\leq L} \rightarrow K$

Why the last encryption step in ECBC-MAC and NMAC?

NMAC: suppose we define a MAC $I = (S,V)$ where

$$S(k,m) = \text{cascade}(k, m)$$

- ☐ This MAC is secure
- ☐ This MAC can be forged without any chosen msg queries
- ☐ This MAC can be forged with one chosen msg query
- ☐ This MAC can be forged, but only with two msg queries

Why the last encryption step in ECBC-MAC?

Suppose we define a MAC $I_{\text{RAW}} = (S, V)$ where

$$S(k, m) = \text{rawCBC}(k, m)$$

Then I_{RAW} is easily broken using a 1-chosen msg attack.

Adversary works as follows:

- Choose an arbitrary one-block message $m \in X$
- Request tag for m . Get $t = F(k, m)$
- Output t as MAC forgery for the 2-block message $(m, t \oplus m)$

Indeed: $\text{rawCBC}(k, (m, t \oplus m)) = F(k, F(k, m) \oplus (t \oplus m)) = F(k, t \oplus (t \oplus m)) = t$

ECBC-MAC and NMAC analysis

Theorem: For any $L > 0$,

For every eff. q -query PRF adv. A attacking F_{ECBC} or F_{NMAC}
there exists an eff. adversary B s.t.:

$$\text{Adv}_{\text{PRF}}[A, F_{\text{ECBC}}] \leq \text{Adv}_{\text{PRF}}[B, F] + 2q^2 / |X|$$

$$\text{Adv}_{\text{PRF}}[A, F_{\text{NMAC}}] \leq q \cdot L \cdot \text{Adv}_{\text{PRF}}[B, F] + q^2 / 2|K|$$

CBC-MAC is secure as long as $q \ll |X|^{1/2}$

NMAC is secure as long as $q \ll |K|^{1/2}$ (2^{64} for AES-128)

An example

$$\text{Adv}_{\text{PRF}}[A, F_{\text{ECBC}}] \leq \text{Adv}_{\text{PRP}}[B, F] + 2q^2 / |X|$$

q = # messages MAC-ed with k

Suppose we want $\text{Adv}_{\text{PRF}}[A, F_{\text{ECBC}}] \leq 1/2^{32} \iff q^2 / |X| < 1/2^{32}$

- AES: $|X| = 2^{128} \implies q < 2^{48}$

So, after 2^{48} messages must, must change key

- 3DES: $|X| = 2^{64} \implies q < 2^{16}$

The security bounds are tight: an attack

After signing $|X|^{1/2}$ messages with ECBC-MAC or
 $|K|^{1/2}$ messages with NMAC
the MACs become insecure

Suppose the underlying PRF F is a PRP (e.g. AES)

- Then both PRFs (ECBC and NMAC) have the following extension property:

$$\forall x, y, w: F_{\text{BIG}}(k, x) = F_{\text{BIG}}(k, y) \Rightarrow F_{\text{BIG}}(k, \mathbf{x||w}) = F_{\text{BIG}}(k, \mathbf{y||w})$$

The security bounds are tight: an attack

Let $F_{\text{BIG}}: \mathbf{K} \times \mathbf{X} \rightarrow \mathbf{Y}$ be a PRF that has the extension property

$$F_{\text{BIG}}(k, x) = F_{\text{BIG}}(k, y) \quad \Rightarrow \quad F_{\text{BIG}}(k, \mathbf{x} \parallel \mathbf{w}) = F_{\text{BIG}}(k, \mathbf{y} \parallel \mathbf{w})$$

Generic attack on the derived MAC:

step 1: issue $|\mathbf{Y}|^{1/2}$ message queries for rand. messages in $\mathbf{X}$.

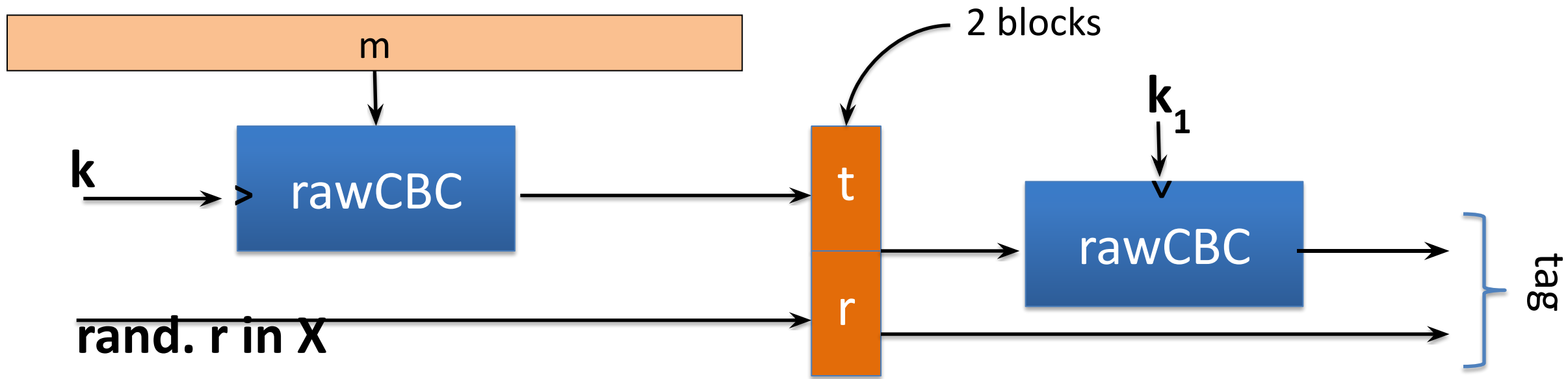
obtain (m_i, t_i) for $i = 1, \dots, |\mathbf{Y}|^{1/2}$

step 2: find a collision $t_u = t_v$ for $u \neq v$ (one exists w.h.p by b-day paradox)

step 3: choose some w and query for $t := F_{\text{BIG}}(k, \mathbf{m}_u \parallel \mathbf{w})$

step 4: output forgery $(\mathbf{m}_v \parallel \mathbf{w}, t)$. Indeed $t := F_{\text{BIG}}(k, \mathbf{m}_v \parallel \mathbf{w})$

Better security: a rand. construction



Let $F: K \times X \rightarrow X$ be a PRF. Result: MAC with tags in X^2 .

Security: $\text{Adv}_{\text{MAC}}[A, I_{\text{RCBC}}] \leq \text{Adv}_{\text{PRP}}[B, F] \cdot (1 + 2q^2 / |X|)$

$\Rightarrow$ For 3DES: can sign $q=2^{32}$ msgs with one key

Comparison

ECBC-MAC is commonly used as an AES-based MAC

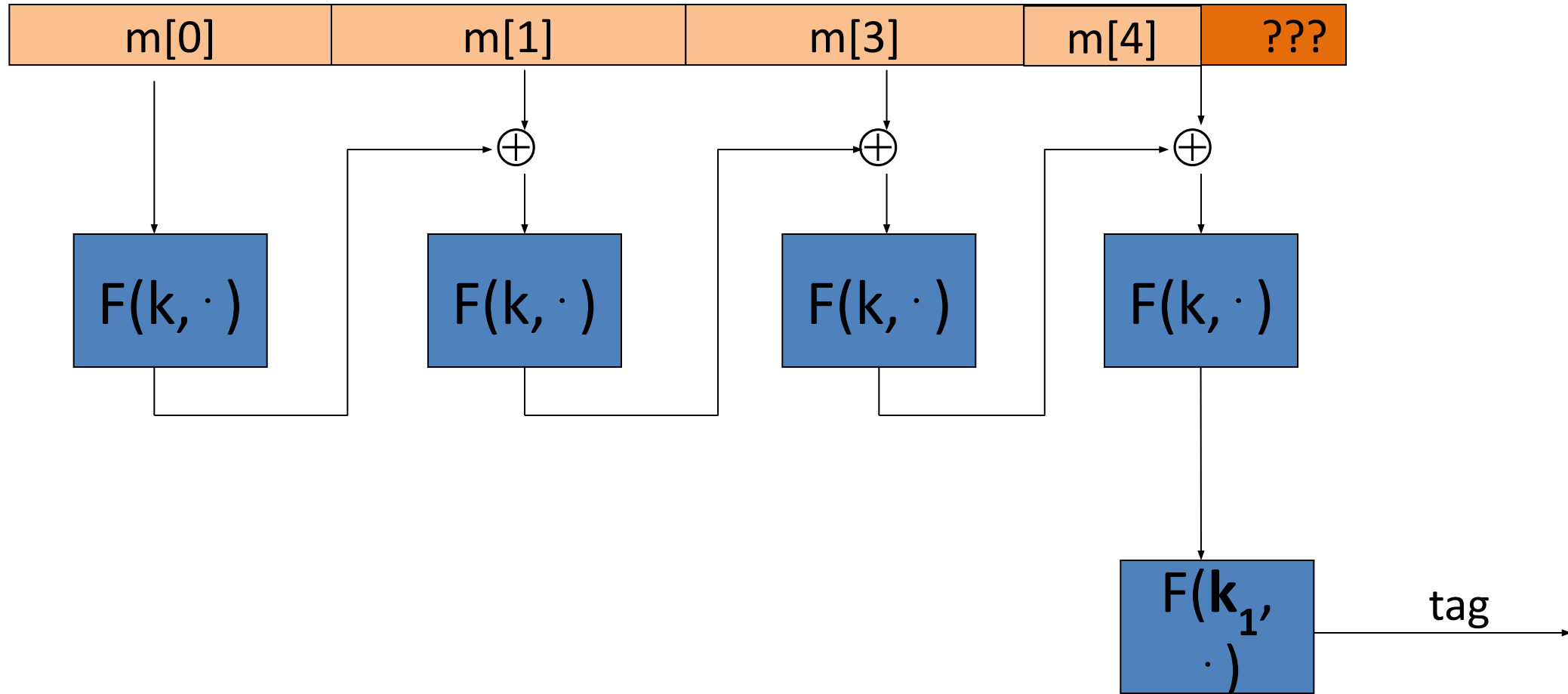
- CCM encryption mode (used in 802.11i)
- NIST standard called CMAC

NMAC not usually used with AES or 3DES

- Main reason: need to change AES key on every block
requires re-computing AES key expansion
- But NMAC is the basis for a popular MAC called HMAC (next)

What about padding?

What if msg. len. is not multiple of block-size?



CBC MAC padding

Bad idea: pad m with 0's



Is the resulting MAC secure?

- ☐ Yes, the MAC is secure
- ☐ It depends on the underlying MAC
- ☐ No, given tag on msg **m** attacker obtains tag on **$m||0$**
- ☐

Problem: $\text{pad}(m) = \text{pad}(m||0)$

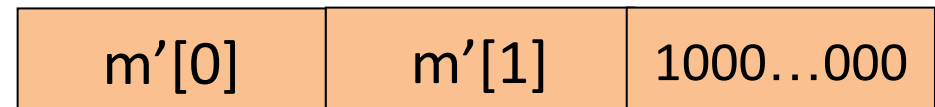
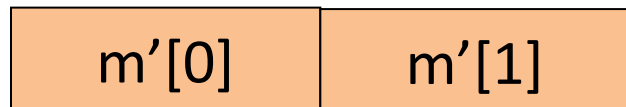
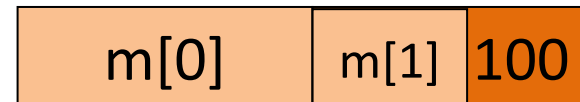
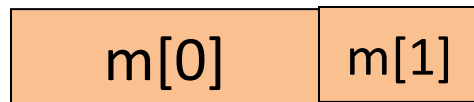
CBC MAC padding

For security, padding must be invertible !

$$\text{len}(m_0) \neq \text{len}(m_1) \Rightarrow \text{pad}(m_0) \neq \text{pad}(m_1)$$

ISO: pad with “1000...00”. Add new dummy block if needed.

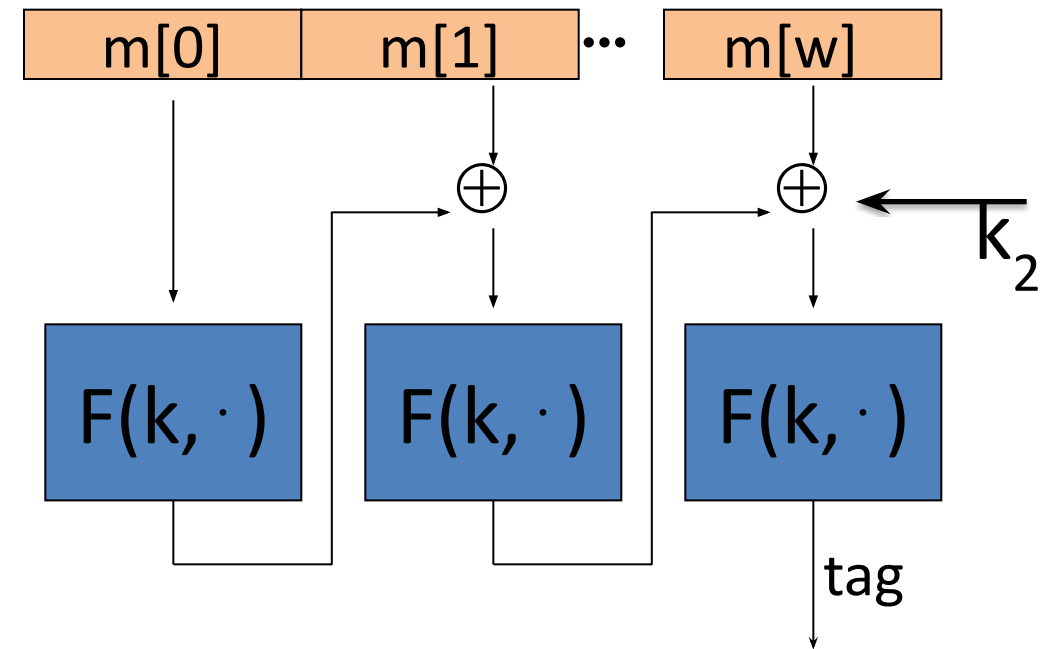
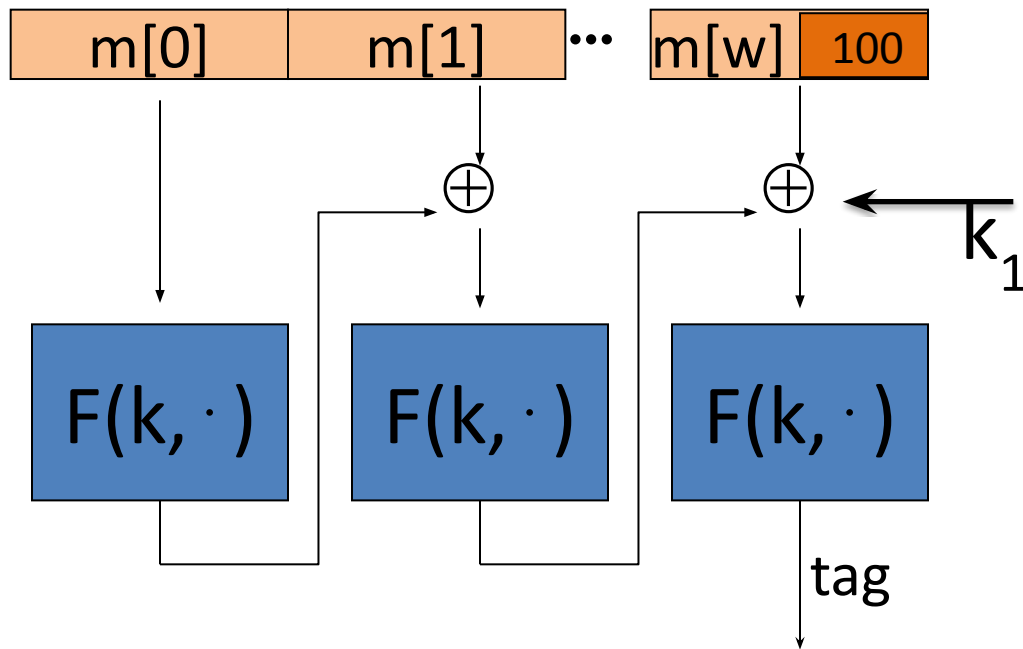
- The “1” indicates beginning of pad.



CMAC (NIST standard)

Variant of CBC-MAC where $\text{key} = (k, k_1, k_2)$

- No final encryption step (extension attack thwarted by last keyed xor)
- No dummy block (ambiguity resolved by use of k_1 or k_2)



More MACs - More Fun!

- PMAC - parallel MAC computation!
- One-time MAC / Many-time MAC
- Carter-Wegman MAC
- ... and many more

but we still didn't talk about the extremely common HMAC (Hash MAC)



Hashes and Resistance

Cryptographic Hash Functions

A Cryptographic Hash Function (CHF) is an algorithm that maps an arbitrary binary string to a string of n bits. $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$

- Message space much larger than output space
 $H: M \rightarrow T, |M| \gg |T|$
- Given the output, we want the input to remain secret and also make it hard for other inputs to get the same output (collision).
- Applications: everywhere (from storing passwords to commitment protocols)

Hash Function Properties

Let $H: M \rightarrow T$, $|M| \gg |T|$

- **Pre-image resistance.** H is pre-image resistant if given a hash value h , it should be difficult to find any message m such that $H(m) = h$. In other words, $P[H(\text{random } m) = h] = 1/|T|$.
- **Second pre-image resistance (weak collision resistance).** H is second-preimage resistant if given a message m_1 , it should be difficult to find a different m_2 such that $H(m_1) = H(m_2)$.
- **(Strong) Collision resistance.** H is collision resistant if it is difficult to find any two different messages m_1 and m_2 such that $H(m_1) = H(m_2)$.

Collision Resistance =>
Second-preimage Resistance

Second-preimage Resistance => Preimage Resistance?

*only true under certain conditions ($|M| \gg |T|$)

Collision Resistance Definition

Let $H: M \rightarrow T$ be a hash function $(|M| \gg |T|)$

A **collision** for H is a pair $m_0, m_1 \in M$ such that:

$$H(m_0) = H(m_1) \quad \text{and} \quad m_0 \neq m_1$$

A function H is **collision resistant** if for all (explicit) “eff” algs. A :

$$\text{Adv}_{\text{CR}}[A, H] = \Pr[A \text{ outputs collision for } H]$$

is “neg”.

Example: SHA-256 (outputs 256 bits)

Ευχαριστώ και καλή μέρα εύχομαι!

Keep hacking!